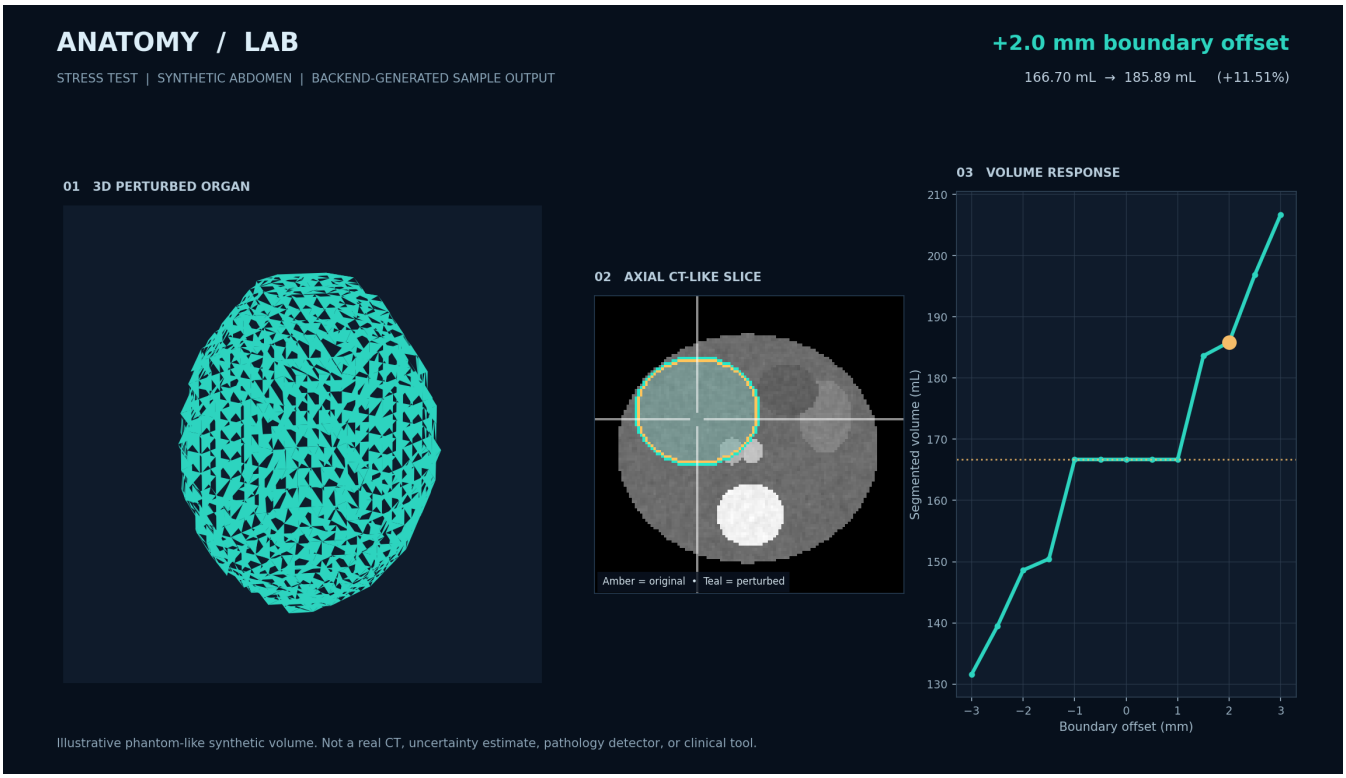
ANATOMY STRESS TEST

A CT geometry experiment you can actually run

A complete, local web application plus an explanation of the mathematics, source code, testing, and responsible public presentation.



Sample generated by the project backend. All anatomy shown is synthetic; this is not a screenshot of the browser interface.

> **THE ONE-SENTENCE IDEA**
>
> Take an existing 3D organ segmentation, expand or erode its boundary by a few millimeters, and measure how much the calculated volume changes.

What is included

- FastAPI local server with an interactive browser interface, bundled 3D plotting, and synchronized axial/coronal/sagittal slices.
- A deterministic synthetic abdomen - no download of a medical dataset and no patient data required.
- Physical-unit boundary offsets, volume response plot, CSV export, tests, README, and LinkedIn copy.

01 / Run the project locally

From ZIP file to working app

The fastest path uses the bundled synthetic demo. You only need Python 3.11+; a network connection is needed the first time you install Python packages. After installation, the app works locally and loads its JavaScript from your own machine.

macOS or Linux

```
unzip Anatomy_Stress_Test.zip
cd Anatomy_Stress_Test
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python server.py
```

Windows PowerShell

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python server.py
```

OPEN THE LOCAL APP

In your browser visit http://127.0.0.1:8000. You should see a synthetic CT-like abdominal viewer, organ selection, boundary slider, and volume-response curve. Press Ctrl+C in Terminal to shut it down.

Your first five clicks

- Select Liver. Its unmodified synthetic mask is the baseline.
- Set the boundary offset to +2.0 mm; compare the amber original edge against the teal altered edge.
- Set the offset to -2.0 mm; observe erosion and the change in volume.
- Click a 2D slice or drag the slice-index slider to synchronize the three cross-sectional views.
- Select Export results (CSV) to save the 13-point sensitivity curve.

02 / What you built

One application, five scientific components

This is a measurement-focused implementation, not just an animated organ model. Each screen element comes from a corresponding computation.

01. Synthetic CT volume	A 3D array of illustrative grayscale CT-like values and eight independent binary organ masks. No diagnostic meaning.
02. Segmentation masks	Each voxel is either inside (1) or outside (0) the selected organ.
03. Physical offsets	A SciPy Euclidean distance transform expands or erodes the mask according to millimeters of voxel-center distance.
04. Quantitative measurements	Volumes are counted on the binary voxel grid and converted from cubic millimeters to milliliters.
05. Visualization and export	A surface mesh illustrates the shape; orthogonal slices show boundary changes; a chart and CSV expose all results.

How it relates to the uploaded Word guide

Your supplied CT Explorer guide builds a Next.js / React Three Fiber application from TotalSegmentator masks. Here, the goal was to provide a single downloadable, runnable project quickly. The application is a new FastAPI + vanilla JavaScript + Plotly implementation, not an edit to that Next.js project. It preserves the concept of 3D anatomy and three slice views but adds a quantitative sensitivity experiment.

Important distinction

> **WHAT THE APP DOES NOT DO**
>
> It does not segment a CT from scratch, learn medical anatomy from AI, detect disease, estimate actual human/algorithm segmentation errors, model blood flow, or provide clinical guidance. The existing masks are its inputs.

03 / Architecture explained

What happens after you move the slider

The frontend requests a new offset from the Python server. The same binary mask drives both the measurements and the visualization, with volume always computed before mesh simplification.

DATA FLOW

Browser control (+2 mm) → FastAPI endpoint → distance-transform mask update → full-resolution voxel count → 3D mesh + slice boundary + line-chart position → browser display

server.py	HTTP routes for /api/meta, /api/metrics, /api/curve, /api/scene, and /api/slice. It caches expensive derived geometry.
anatomy_lab/dataset.py	Generates the synthetic volume or, if configured, reads a local ct.nii.gz with separate segmentation mask NIfTIs.
anatomy_lab/geometry.py	Crops a region around each organ, calculates distance transforms, evaluates offsets, counts voxels and extracts meshes.
anatomy_lab/imaging.py	Applies grayscale CT windowing, oriented axial/coronal/sagittal views, and colored boundary overlays.
static/app.js	Connects control sliders, coordinates all three slice views, downloads CSV, and draws 3D objects and the sensitivity curve.
tests/	Checks the math, orientation, mesh existence, and HTTP responses.

Why crop the masks?

Distance transforms on an entire clinical CT can consume substantial memory. The app first finds the organ bounding box and adds a margin wider than the maximum 3 mm offset. The expensive calculations then happen within this padded region of interest. Volumetry still uses original-grid voxels within the affected region.

Why use a surface mesh at all?

The 3D display is made of triangles extracted with marching cubes. For speed, its triangles can be simplified. The mesh is never used to report volume. That avoids confusing rendering quality with quantitative accuracy.

04 / The core mathematics

How a 2 mm shift becomes a volume change

Imagine every voxel center is a point in 3D space. The segmentation identifies which points belong to an organ. The app measures physical distances between these centers rather than merely counting pixels.

1. Physical voxel volume

VOLUME CALCULATION

If voxel spacing is s_x, s_y, s_z in millimeters, then $V\,[mL] = N \times s_x \times s_y \times s_z / 1000$, where N is the number of included voxels. One cubic millimeter equals 0.001 mL.

2. Expansion: positive boundary offset

At +d mm, include every original organ voxel and all previously excluded voxel centers whose shortest Euclidean distance to an included voxel center is no greater than d.

3. Erosion: negative boundary offset

At -d mm, retain only original organ voxel centers whose shortest Euclidean distance to an excluded voxel center is strictly greater than d. This removes a thin exterior layer of the mask.

4. Compare the result with the baseline

PERCENT CHANGE

Relative change [%] = 100 × (edited volume / original volume - 1). Because voxels enter or leave in whole increments, the response may look like a staircase; there is no guarantee that moving a slider by 0.5 mm will change the count.

A concrete output from the synthetic liver

In the bundled deterministic sample, the baseline is about 166.70 mL. With a +2.0 mm expansion, the result is about 185.89 mL, or +11.51%. This is a deliberately fabricated educational organ shape. The percentage is not a measured patient error rate and does not establish the magnitude of real-world segmentation uncertainty.

Why not simply scale every organ by 2%?

A change of 2 mm is an absolute geometric displacement, not a percentage. Small, irregular, or high-surface-area structures may exhibit different relative volume sensitivity than large smooth structures. Voxel spacing and sampling also affect the measured response.

05 / Reading the viewer correctly

What the 3D mesh and slices are showing

A CT is a 3D grid. The viewer extracts three perpendicular cross-sections around a shared crosshair. It transforms canonical RAS+ voxel coordinates to common radiological display conventions.

Axial	The patient is viewed from the feet. Patient right appears on the image left; patient anterior is at the image top.
Coronal	Facing the patient. Patient right is left on screen; superior is at the image top.
Sagittal	Side view. Anterior is image left; superior is at the top.

Colors and geometry

- Amber outline: the original binary segmentation boundary.
- Teal outline and surface: the boundary after the current offset.
- White crosshair: a common 3D voxel location shared across slice views.
- Ghosted neighboring organs: extra synthetic context objects; not used in this measurement.

Two different forms of resolution

The server loads measurements from the original voxel grid. The browser uses lower-detail geometry for responsive 3D interaction. A convincing 3D appearance does not prove the measurement is correct, so the math has its own tests.

Loading a real scan

```
# From the project folder:
export ANATOMY_DATASET_DIR="/absolute/path/to/subject"
python server.py
```

The expected subject folder contains ct.nii.gz and segmentations/organ_name.nii.gz. The loader reads NIfTI via nibabel, reorients both to closest canonical RAS+, and rejects masks whose affine or shape does not align with the CT. It does not silently resample images.

> **PRIVACY CHECK**
>
> The repository contains no clinical CT files. Do not upload patient scans, DICOM headers, subject identifiers, or potentially identifying screenshots to public GitHub or LinkedIn.

06 / What was actually checked

Tests, reliability, and limitations

The shipped code was exercised using its deterministic synthetic test dataset. Its automated checks completed successfully in the creation environment.

VERIFIED IN THIS BUILD

14 automated tests passed using pytest. The tests covered zero-offset identity, monotonicity of voxel count, unit conversion, physical-space mesh output, radiological slice orientation, PNG generation, and HTTP endpoints. The JavaScript file also passed a syntax check with Node.

What is tested - and why

Zero offset	The original segmentation and volume must be reproduced exactly.
Monotonic response	Moving from negative to positive offsets must never reduce the voxel count.
Milliliter units	The actual physical volume must equal voxel count times voxel spacing product.
Orientation	Anatomical sides must not silently flip during 2D display.
HTTP API	The web server returns structured measurements, meshes, and actual PNG bytes.

What remains unverified

- The environment could not install nibabel because external package downloads were unavailable; the optional real-NIfTI loader has not been run end-to-end here.
- Automated browser navigation was restricted by the environment. HTTP routes, static files, backend sample output, and JavaScript syntax were checked, but full interactive browser rendering was not visually confirmed here.
- No clinical validation, interobserver segmentation study, or external dataset benchmark was conducted.

SCIENTIFIC INTERPRETATION

The slider perturbs a mask according to a geometric rule. It does not quantify uncertainty of a segmentation model or establish typical radiologist contour variability. Those questions require measured error distributions and independent annotated data.

07 / Share the project responsibly

GitHub + LinkedIn publishing checklist

First, run the local app yourself and confirm that the expected baseline and expanded volumes appear. Then make a short screen recording or screenshot using the bundled synthetic demo.

Before publishing

- Check git status. The supplied .gitignore excludes common clinical image formats and a data/ directory; manually verify nothing private is staged.
- Run python -m pytest -q from the project directory after installing requirements-dev.txt.
- Edit the README if you alter assumptions, add datasets, or change the code. The MIT license covers the new project code; original datasets retain their own licensing terms.

Initial Git commands

```
git init
git add .
git commit -m "Build Anatomy Stress Test"
git branch -M main
# Create an empty GitHub repository first, then:
git remote add origin https://github.com/YOUR_USER/anatomy-stress-test.git
git push -u origin main
```

A concise LinkedIn caption

DRAFT - USE ONLY AFTER VERIFYING LOCALLY

I built Anatomy Stress Test: a local, open-source experiment showing how small artificial shifts in a 3D segmentation boundary change voxel-based organ-volume measurements. It combines a 3D viewer, linked CT-like slices, a millimeter-scale offset control, and a quantitative response curve. The public demonstration uses synthetic data. This is an educational sensitivity tool, not a clinical device or a validated uncertainty model. [Your GitHub URL]

What can you accurately claim?

Yes: built a complete application, used voxel spacing and Euclidean distance transforms, visualized an artificial sensitivity experiment, and included reproducible tests. No: invented a new mathematical method, validated segmentation error in patients, improved diagnostic accuracy, or delivered a clinical-grade device.

Next experiment after this quick project

A strong incremental extension would be distance-to-neighbor sensitivity: compute the minimum surface-to-surface separation between two existing masks and test how that estimate changes under carefully defined perturbations. As with this tool, the research contribution would depend on validation and comparison to prior art, not the interface alone.

08 / Deeper engineering reference

A map for studying the repository

Read these files in this order. The goal is to understand each transformation from original input to displayed measurement, rather than memorizing the Python syntax.

Start with geometry.py	Find StressExperiment.from_mask. It crops the mask and precomputes distances in physical millimeters.
Then offset_mask	Trace how negative and positive offsets select voxels. Check that offset 0 returns exactly the original.
Then metrics + curve	Notice that volumes are computed as included voxel counts, independently of the mesh.
Then imaging.py	Observe how the CT is windowed to grayscale, how borders are found, and how RAS+ arrays are flipped into standard displayed planes.
Then server.py	Follow an HTTP request: /api/metrics, /api/slice or /api/scene.
Finally app.js	Study how a slider triggers API requests and updates the 3D figure, 2D slices, and curve.

Learning exercise: predict before you run

- Why can a +0.5 mm boundary offset yield exactly zero volume change?
- Why might changing a thin vessel by 2 mm affect its fractional volume more than a large organ?
- What happens if an image spacing is 0.7 × 0.7 × 5.0 mm instead of isotropic 1 mm?
- Why is it necessary to compare image and mask affines instead of comparing only array shapes?

BASIC ANSWERS

Voxel inclusion is discrete; geometry controls relative surface-to-volume changes; anisotropic sampling creates directional quantization; and arrays with the same shape may represent different physical positions or orientations.

Project attribution and sources

Conceptual inspiration: user-supplied Build a 3D CT scan explorer with one prompt by @hinnazeejah. NIfTI convention: nibabel; morphological distance-transform implementation: SciPy; 3D rendering: Plotly; surface extraction: scikit-image marching cubes. TotalSegmentator data and attribution: https://zenodo.org/records/10047292 (data not bundled). All results here use a separately generated synthetic array.